

Competitive Exam Practice 7: Doubly & Circular Lists, List-Backed Stack and Queue, and Polynomials

GATE (Computer Science) Pattern

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 3, 2026

Each question carries 1–2 marks. MCQ = single correct option; NAT = Numerical Answer Type.

Q1 [*GATE CSE 1997 Q1.4, verified — GATEOverflow V2 p.233*] [**MCQ, 1 mark**] The concatenation of two lists is to be performed in $O(1)$ time. Which of the following implementations of a list should be used?

- (A) Singly linked list
- (B) Doubly linked list
- (C) Circular doubly linked list
- (D) Array implementation of list

Q2 [*GATE CSE 2004 Q36, verified — GATEOverflow V2 p.235*] [**MCQ, 1 mark**] A circularly linked list is used to represent a queue. A single variable **p** is used to access the queue. To which node should **p** point such that both the operations **enqueue** and **dequeue** can be performed in constant time?

- (A) rear node
- (B) front node
- (C) not possible with a single pointer
- (D) node next to front

Q3 [*GATE CSE 2017 Set 2 Q13, verified — GATEOverflow V2 p.242*] [**MCQ, 2 marks**] A circular queue has been implemented using a singly linked list where each node consists of a value and a single pointer pointing to the next node. We maintain exactly two external pointers **FRONT** and **REAR** pointing to the front node and the rear node of the queue, respectively. Which of the following statements is/are CORRECT for such a circular queue, so that insertion and deletion operations can be performed in $O(1)$ time?

- I. Next pointer of front node points to the rear node.
- II. Next pointer of rear node points to the front node.

- (A) (I) only
- (B) (II) only

- (C) Both (I) and (II)
- (D) Neither (I) nor (II)

Q4 [*GATE CSE 2018 Q3, verified — GATEOverflow V2 p.242*] [MCQ, 1 mark] A queue is implemented using a non-circular singly linked list. The queue has a head pointer and a tail pointer. Let n denote the number of nodes in the queue. Let **enqueue** be implemented by inserting a new node at the head, and **dequeue** be implemented by deletion of a node from the tail. Which one of the following is the time complexity of the most time-efficient implementation of **enqueue** and **dequeue**, respectively, for this data structure?

- (A) $\Theta(1)$, $\Theta(1)$
- (B) $\Theta(1)$, $\Theta(n)$
- (C) $\Theta(n)$, $\Theta(1)$
- (D) $\Theta(n)$, $\Theta(n)$

Q5 [*GATE CSE 2023 Q3, verified — GATEOverflow V2 p.237*] [MCQ, 1 mark] Let **SLLdel** be a function that deletes a node in a singly linked list given a pointer to the node and a pointer to the head of the list. Similarly, let **DLLdel** be another function that deletes a node in a doubly linked list given a pointer to the node and a pointer to the head of the list. Let n denote the number of nodes in each of the linked lists. Which one of the following choices is TRUE about the worst-case time complexity of **SLLdel** and **DLLdel**?

- (A) **SLLdel** is $O(1)$ and **DLLdel** is $O(n)$
- (B) Both **SLLdel** and **DLLdel** are $O(\log n)$
- (C) Both **SLLdel** and **DLLdel** are $O(n)$
- (D) **SLLdel** is $O(n)$ and **DLLdel** is $O(1)$

Q6 [*Original, GATE-pattern*] [MCQ, 2 marks] A doubly linked list holds **head** $\leftrightarrow 7 \leftrightarrow 19 \leftrightarrow 42 \leftrightarrow \text{NULL}$. A pointer **p** points at the node holding 19. That node is deleted by the deck's delete-by-pointer and then **free(p)** is called. Which statement is TRUE about the resulting list and the cost?

- (A) The list becomes **head** $\leftrightarrow 7 \leftrightarrow 42$; 7's **next** is set to 42 and 42's **prev** is set to 7, both reached in $O(1)$ from **p**; the delete is $O(1)$ worst case once **p** is known.
- (B) The list becomes **head** $\leftrightarrow 7 \leftrightarrow 42$, but the predecessor of the deleted node must first be found by a walk, so the delete is $O(n)$ worst case.
- (C) The list is unchanged — **head** $\leftrightarrow 7 \leftrightarrow 19 \leftrightarrow 42$ — because delete-by-pointer only moves **head**.
- (D) The list becomes **head** $\leftrightarrow 7 \leftrightarrow 42$, but only 7's **next** is updated; 42's **prev** still points at the freed node, breaking the two-link invariant.

Q7 [*Original, GATE-pattern*] [MCQ, 2 marks] A doubly linked list holds **head** $\leftrightarrow 3 \leftrightarrow 8 \leftrightarrow 14 \leftrightarrow \text{NULL}$, and **p** points at the node holding 8. A new node holding 5 is inserted immediately after **p**. Which sequence of pointer writes restores the two-link invariant and yields **head** $\leftrightarrow 3 \leftrightarrow 8 \leftrightarrow 5 \leftrightarrow 14$?

- (A) `newNode->next = p->next;`
`newNode->prev = p;`
`if (p->next != NULL) p->next->prev = newNode;`
`p->next = newNode;`
- (B) `p->next = newNode;`
`newNode->prev = p;`
`newNode->next = p->next;`
- (C) `newNode->next = p->next;`
`p->next = newNode;`
`newNode->prev = p;`
`if (p->next != NULL) p->next->prev = newNode;`
- (D) `newNode->prev = p;`
`newNode->next = p->next;`
`p->next = newNode;`
`p->next->prev = newNode;`

Q8 [Original, GATE-pattern] [MCQ, 1 mark] A singly circular list reads `head` $\rightarrow$ `p` $\rightarrow$ `q` $\rightarrow$ `r` $\rightarrow$ `p`. The following loop is run:

```
ptr = head;
while (ptr != NULL) {
    visit(ptr);
    ptr = ptr->next;
}
```

What happens?

- (A) It visits `p`, `q`, `r` and then stops when `ptr` becomes `NULL`.
- (B) It loops forever, cycling `p`, `q`, `r`, `p`, `q`, `r`, ... and never meets `NULL`.
- (C) It visits only the node `p`, then stops at the next `head`.
- (D) It aborts after three visits because the list has no end.

Q9 [Original, GATE-pattern] [MCQ, 2 marks] Two sparse polynomials are stored as ordered (coeff, exp) chains in strictly descending exponent order: $A = 6x^3 + 2x + 9$ and $B = 4x^3 + 5x^2 - 2x$. Using the deck's merge-by-descending-exponent addition, which result and cost are correct?

- (A) $A + B = 10x^3 + 5x^2 + 9$, produced by one merge pass costing $O(n + m)$ worst case; the cancelled x term is never stored.
- (B) $A + B = 10x^3 + 5x^2 + 0x + 9$; the merge stores the zero term, still costing $O(n + m)$.
- (C) $A + B = 10x^3 + 5x^2 + 9$, but the merge costs $O(n \cdot m)$ because every pair of terms is multiplied.
- (D) $A + B = 10x^3 + 5x^2 + 4x + 9$, produced by one merge pass costing $O(n + m)$.

Q10 [Original, GATE-pattern] [MCQ, 1 mark] A linked-list stack has `head` $\rightarrow$ 7 $\rightarrow$ 14 $\rightarrow$ 21 $\rightarrow$ `NULL`, with the top of the stack at `head`. `push(28)` is performed, then one `pop()`. Which statement is TRUE?

- (A) `push` links 28 in front, giving `head` $\rightarrow 28 \rightarrow 7 \rightarrow 14 \rightarrow 21$; `pop()` returns 28 and the new top is 7; both operations are $O(1)$ worst case.
- (B) `push` appends 28 at the tail, giving `head` $\rightarrow 7 \rightarrow 14 \rightarrow 21 \rightarrow 28$; `pop()` removes the front, so it returns 7.
- (C) `push` links 28 in front, giving `head` $\rightarrow 28 \rightarrow 7 \rightarrow 14 \rightarrow 21$; `pop()` returns 7 because a stack removes the oldest item.
- (D) Both `push` and `pop` take $O(n)$ worst case, because a linked list has no indices.